67.101
תש"ע / א

האוניברסיטה העברית

בי"ס להנדסה ולמדעי המחשב

בחינה בקורסי מבוא למדעי המחשב:

67101 – מבוא למדעי המחשב

67108 - מבוא למדעי המחשב לתלמידי תלפיות

67112 - מבוא למדעי המחשב לתלמידי תלפיות מצומצם

מועד א' - סמסטר א' - תש"ע – 2009-2010 **תאריך:** 4.2.2010

פרופ' דפנה ויינשל, פרופ' יובל רבני, מר אלעד מזומן **משך הבחינה:** שלוש שעות

הנחיות:

1. יש לענות על 3 מתוך 4 השאלות הבאות (3 בלבד). משקל כל השאלות זהה (33 נק').
2. **יש לכתוב פתרון לכל אחת מהשאלות שבחרת <u>במחברת נפרדת</u>. יש לציין על כריכת המחברת לאיזה שאלה המחברת מתייחסת, באופן קריא וברור.**
3. יש לציין את מס' תעודת הזהות ומספר המחברת על **כל** מחברת.
4. הקפידו הקפדה יתרה על סדר וכתב יד קריא. בחינה בלתי קריאה לא תזכה את כותבה במלוא הנקודות.
5. יש למחוק טיוטות באופן ברור.
6. **יש להשאיר שוליים.**
7. **אסור להשתמש בעט אדום.**
8. הקפידו על סגנון תכנותי נאות: כתיבה באופן מודולרי, ללא שיכפול קוד וכו'.
9. תעדו כל תכנית כך שניתן יהיה להבינה בנקל (מותר לתעד בעברית).
10. פתרון מיטבי לשאלה יחשב פתרון **פשוט ויעיל ככל האפשר**. תוכנית מסורבלת או מסובכת לא תתקבל כתשובה מלאה (גם אם היא נכונה).
11. **אין להקצות** או להשתמש במבני נתונים פרט לאלו שהוגדרו בכל שאלה. עם זאת, מותר להשתמש במבני נתונים נוספים אשר גודלם ומספרם אינו תלוי בגודל הקלט (למשל מותר להקצות מערך בן שלשה תאים פעם אחת).
12. בכל השאלות ניתן להניח שהקלט חוקי ומקיים את תנאי השאלה. כמו כן, ניתן להניח כי המערכים והרשימות המקושרות הנתונות מכילים ערכים תקינים ואינם NULL, או מערכים בגודל 0, אלא אם מובהר בשאלה אחרת.

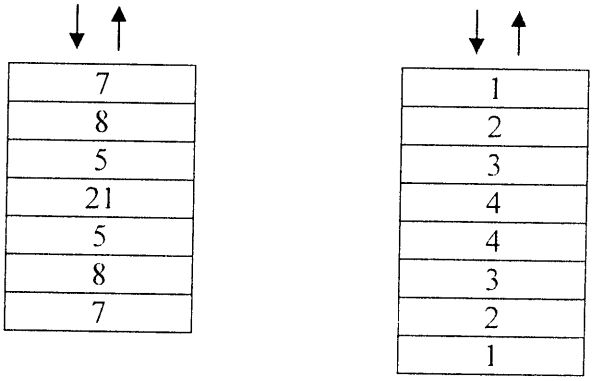
67-101
א/תשע"ח

שאלה 1 (33 נקודות)

מחסנית ותור פלינדרום

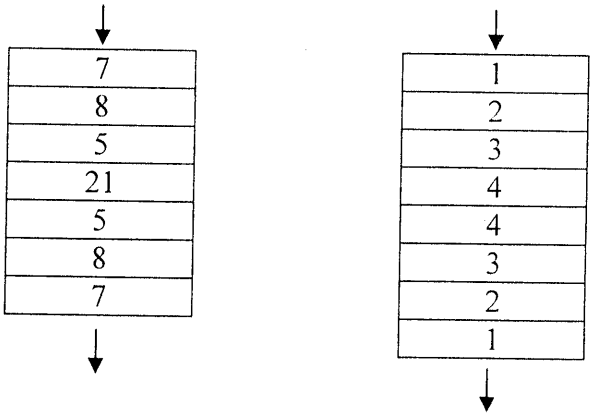
מחסנית פלינדרום הינה מחסנית אשר הינה סימטרית מבחינת הערכים בה.

לדוגמא, המחסניות הבאות הינן מחסניות פלינדרום (החיצים מסמנים את ראש המחסנית):



באופן דומה, תור פלינדרום הינו תור אשר סימטרי מבחינת הערכים בו.

לדוגמא, התורים הבאים הינן תורי פלינדרום (החיצים מסמנים את כיוון הכניסה/יציאה של איברים מהתור):



נתונה המחלקה Queue המממשת תור (אין צורך לממש מחלקה זו). מחלקה זו הינה בעלת השיטות הבאות:

`public Queue()`	בונה תור חדש.
`public void enqueue(int value)`	מכניסה את הערך הנתון לסוף התור.
`public int dequeue()`	מוציאה את הערך שבראש התור. אם התור ריק תיגרם שגיאה והתכנית תעצר.

נתונה המחלקה Stack המממשת מחסנית (אין צורך לממש מחלקה זו). מחלקה זו הינה בעלת השיטות הבאות:

`public Stack()`	בונה מחסנית חדשה.
`public void push(int value)`	מכניסה את הערך הנתון למחסנית.
`public int pop()`	מוציאה את הערך שבראש המחסנית. אם המחסנית ריקה תיגרם שגיאה והתכנית תעצר.

(המשך שאלה 1 בעמוד הבא)

67.101
16/מועד

א. (13 נקודות)

עליכם לממש שיטה לא רקורסיבית:

```
public static boolean checkPalindrome1(Queue q1,int numOfElements)
```

המקבלת כקלט תור ואת מספר האיברים שבו. ומחזירה true באם התור הינו תור פלינדרום ו-false אחרת.

ב. (3 נקודות)

מהי סיבוכיות זמן הריצה של השיטה שמימשתם בסעיף א'?

ג. (14 נקודות)

עליכם לממש שיטה לא רקורסיבית:

```
public static boolean checkPalindrome2(Stack s1,int numOfElements)
```

המקבלת כקלט מחסנית ואת מספר האיברים שבה, ומחזירה true באם המחסנית הינה מחסנית פלינדרום ו-false אחרת. בסעיף זה, הינכם רשאים לממש את השיטה באמצעות תור אחד נוסף בלבד (תוך שימוש במחלקה הנתונה Queue).

ד. (3 נקודות)

מהי סיבוכיות זמן הריצה של השיטה שמימשתם בסעיף ג'?

הערות:

1. אינכם נדרש לשמור על שלמות המחסנית/התור שניתנו כקלט.
2. שימו-לב כי אין באפשרותכם להשתמש בשיטה isEmpty() עבור המחסניות/התורים, היות והמחלקות Stack ו-Queue אינן מכילות שיטה זו.
3. כמו בכל שאלה, אינכם רשאים להגדיר מערך או מבני נתונים אחרים התלויים בגודל הקלט, מלבד אלו המצויינים בשאלה.
4. כמו בכל שאלה, הניקוד המלא עבור שאלה זו יינתן רק עבור פתרון המממש את השיטות הנ"ל בסיבוכיות זמן הריצה היעילה ביותר.

67.101
תש"ע/א

שאלה 2 (33 נקודות)

המערך הבא ישמש אותנו בשאלה זו:

arrQ2 =	133	51	5	196	260	100	95

א. (12 נקודות)

נתון הקוד הבא:

```
public static int foo(int[] arr, int someIndex) {
        int p = arr[someIndex], b = 1, t = arr.length-1;
        swapInArray(arr,0, someIndex);
        while (b<=t) {
            while (p>arr[b]) {
                b++;
            }
            while (p<arr[t]) {
                t--;
            }
            if (b<t) {
                swapInArray(arr,b,t);
                b++;
                t--;
            }
        }
        swapInArray(arr, 0, t);
        return t;
    }

public static void swapInArray(int[] arr, int idxA, int idxB) {
        int tmp = arr[idxA];
        arr[idxA] = arr[idxB];
        arr[idxB] = tmp;
    }
```

הסבירו את פעולת השיטה **foo** ואת משמעותה. מה משמעות הפרמטרים שהיא מקבלת? מה משמעות התוצאה שהיא מחזירה?

מהו הערך שהשיטה תחזיר, וכיצד יראה המערך arrQ2 לאחר הקריאה הבאה: foo(arrQ2,6) ?

<u>הערה:</u> שימו לב עד כמה בחירת שמות משתנים טובה משפיעה על קריאות הקוד.

ב. (3 נקודות)

מהי סיבוכיות זמן הריצה של השיטה בסעיף א', כתלות בגודל הפלט? הסבירו בפירוט.

ג. (2 נקודות)

תזכורת: עבור מערך בגודל אי-זוגי, חציון הינו הערך שגדול ממחצית יתר האברים וקטן ממחצית יתר האברים. בכיתה למדנו אלגוריתם למציאת חציון.

מהו החציון במערך arrQ2?

<u>(המשך שאלה 2 בעמוד הבא)</u>

67.101
1א/מ"ן

הערה: לצורך פתרון הסעיפים הבאים הינכם רשאים (אך לא חייבים) להשתמש בשיטה הבאה למציאת חציון:

```
public static int findMedian(int[] arr)
```

הניחו כי שיטה זו ממומשת, ומוצאת את החציון בסיבוכיות זמן ריצה של O(n).

ד. (13 נקודות)

מערך A באורך אי זוגי - 2n+1, הוא מערך גלי אם:

$$A[0] \geq A[1] \leq A[2] \geq A[3] \leq A[4] \ldots \geq A[2n] \leq A[2n+1]$$

או במילים: כל האברים באינדקסים האי-זוגיים (1,3,....2n+1) קטנים משני שכניהם וכל האברים באינדקסים הזוגיים גדולים משני שכניהם. שימו לב שאין דרישה ליחס סדר בין אברים לא שכנים.

לדוגמא, המערך הבא הוא מערך גלי:

100	51	133	5	260	95	196

כיתבו שיטה:

```
public static int[] wigglyArrray(int[] arr)
```

המקבלת כקלט מערך של מספרים שלמים באורך אי-זוגי, ומחזירה מערך חדש (שהוקצה ב-new) ובו אברי מערך הקלט מסודרים מחדש, כך שהפלט הוא מערך גלי.

הערות:

ניתן ורצוי להשתמש בפונקציות מסעיפים קודמים.

ניתן לשנות את סדר אברי המערך שהתקבל בקלט.

הניקוד עבור מימוש השיטה ינתן בהתאם לסיבוכיות הריצה של המימוש – מימוש יעיל יקבל ניקוד גבוה יותר ממימוש לא יעיל.

ה. (3 נקודות)

מהי סיבוכיות זמן הריצה של השיטה בסעיף ד', כתלות בגודל הקלט? הסבירו בפירוט.

67.101
מ"ר/א

<u>שאלה 3</u> (33 נקודות)

א. (20 נקודות)

בפסטיגל מתחרות קבוצות בנות n חברים. להנהלת הפסטיגל יש m סוגי תלבושות. שחקנים הלבושים באותה תלבושת נראים זהים. משיקולי אסטתיקה הנהלת הפסטיגל לא מאפשרת לשני שחקנים עם תלבושת זהה לעמוד אחד ליד השני (הם תמיד עומדים בשורה). משיקולי יצירתיות, הנהלת הפסטיגל לא מוכנה ששתי קבוצות יופיעו על הבמה בסידור תלבושות זהה.

כתבו שיטה רקורסיבית

```
public static void printStaging(int actors,int costumes)
```

המקבלת את actors מספר השחקנים בכל קבוצה ואת costumes מספר סוגי התלבושות השונות הקיימות.

על השיטה **להדפיס** את כל הסידורים האפשריים בלי להתפשר על העקרונות היצירתיים והאסתטיים.

דוגמא: עבור הקריאה:

```
public static void printStaging(3,3)
```

יודפסו הסידורים: **(0,1,0)(0,1,2)(0,2,0)(0,2,1)(1,0,1)(1,0,2)(1,2,0)(1,2,1)(2,0,1)(2,0,2)(2,1,0)(2,1,2)** כאשר כל מספר מציין תלבושת (למשל **0**-סופרמן, **1**- ספיידרמן וכו') וסדר ההדפסות לא משנה.

ב. (13 נקודות)

במהלך החזרות מתגלים קשיי תקציב ולכן לא ניתן לרכוש יותר מ-**k** תלבושות מכל סוג. כתבו כעת שיטה רקורסיבית שניה:

```
public static void printStaging(int actors,int costumes,int k)
```

המקבלת פרמטר נוסף **k** המגדיר את אילוצי ההלבשה. **k** קובע חסם על מספר התלבושות מכל סוג.

שים לב כי הפתרון המיטבי לסעיף זה אינו עובר על כל האפשרויות שהודפסו בסעיף א'.

דוגמא:

עבור **printStaging(8,5,3)**, הסידור **(0,1,2,1,2,3,4,5)** הוא סידור חוקי, בעוד הסידור **(1,2,1,2,1,3,1,4)** אינו חוקי מאחר ויש ארבעה שחקנים עם אותה תלבושת (ספיידרמנים) כאשר להפקה יש רק **k=3** תלבושות ספיידרמן.

הערות:

1. בשני סעיפי שאלה זו מותר להקצות זיכרון על ה-heap. הקצאת הזיכרון תהיה לינארית באורך הקלט.
2. הנכם נדרשים לפתרון רקורסיבי של שני הסעיפים. לכן, הנכם רשאים שהשיטה **printStaging** עצמה לא תהייה רקורסיבית, באם היא קוראת לשיטה רקורסיבית אחרת הממשת את הפתרון.

67.101
מס' 11/11

שאלה 4 (33 נקודות)

א. (5 נקודות)

נתון קטע הקוד הבא :

```
public class Card{
// Card can be taken from the pile
    private static int _numOfTakes = 0;
    protected final int _cardNum;
    protected boolean _taken;

    Card(int cardNum){
        _cardNum=cardNum;
    }
    public int take(){
        _numOfTakes++;
        return _cardNum;
    }
    public int getNumOfTakes(){
        return _numOfTakes;
    }
}
```

מה יודפס אחרי קטע הקוד הבא? נמקו

```
int newCard;
Card card1 = new Card(2);
Card card2 = new Card(5);
newCard = card1.take();
newCard = card1.take();
newCard = card2.take();
System.out.println(card1.getNumOfTakes());
System.out.println(card2.getNumOfTakes());
```

ב. (14 נקודות)

נוסיף את הממשק הבא

```
public Interface Changeable{
    public int change(int changeTo);
}
```

ונוסיף את האובייקטים הבאים

```
public class ChangeableCard extends Card implements Changeable{
...
}
public class TakiCard extends Card{
...
}
public class ChangeableTakiCard extends TakiCard implements
Changeable{
...
}
```

(המשך שאלה 4 בעמוד הבא)

67.101
מועד א

בנוסף, הוגדר קטע הקוד הבא:

```
Changeable c1,c2;
Card cr1,cr2;
ChangeableCard cc1,cc2;
TakiCard tc1,tc2;
ChangeableTakiCard ct1,ct2;
```

עבור כל אחד מתתי הסעיפים הבאים (1-7) – ציינו אם הוא יעבור קומפילציה (כן/לא) ונמקו בשורה אחת מדוע.

```
1. c1 = new Card(1);
2. cr2 = new ChangeableTakiCard(...);
3. tc1 = new Card(5);
4. cr1 = new ChangeableCard(...);
   c1 = cr1;
5. cr2 = new ChangeableCard(...);
   cc2 = (Changeable)cr2;
6. c2 = new ChangeableTakiCard(...)
   tc1 = (TakiCard)c2;
7. tc2 = new TakiCard(...);
   cc1 = (ChangeableCard)tc2;
```

ג. (4 נקודות)

מימוש המחלקה ChangeableCard הוא כדלהלן :

```
public class ChangeableCard  extends Card implements Changeable{
    ChangeableCard(int cardNum){
        super(cardNum);
    }
    public int change(int changeTo){
        _cardNum = changeTo;
    }
}
```

האם קטע הקוד הבא יעבוד ? אם כן - נמקו ורשמו מה יודפס ומדוע , אם לא - נמקו מדוע ורשמו מה צריך לעשות כדי לתקן אותו.

```
ChangeableCard c1 = new ChangeableCard(4);
c1.changeTo(5);
System.out.print(c1.take());
```

(המשך שאלה 4 בעמוד הבא)

67.101
מועד א'/ 16

ד. (10 נקודות)

המחלקה MagicCard יורשת את ChangeableCard. האובייקט mc הוא מופע שלה והמימוש של המחלקה נתון לבחירתכם .
בנוסף המחלקה ChangeableCard מממשת גם את המתודה הבאה :

```
public int compareWith(ChangeableCard c);
```

עבור כל אחד מתתי הסעיפים הבאים (1-5) הבאים ציינו האם ניתן לממש את המחקלה MagicCard כך שמקטע הקוד יתקמפל וירוץ. אם כן תארו כיצד. אם לא הסבירו מדוע.

```
1. int x = 7;
   mc.change(x);
2. double x = 1.5;
   mc.change(x);
3. Changeable c = new MagicCard(10);
   double x = 1.5;
   c.change(x);
4. Card c = new Card(5)
   mc.compareWith(c);
5. Changeable ch = new ChangeableCard(...);
   ChangeableCard cc = new MagicCard(...);
   cc.compareWith(ch);
```